

OpenCV + PyTorch CNN + YOLO 8天课件

课程定位

本课程面向已经掌握 Python 基础语法、函数、面向对象、文件读写的学习者。课程目标是用 8 天完成一条完整计算机视觉学习路线：

- 前 3 天：OpenCV 图像处理基础、视频处理、传统视觉特征与分割。
- 后 5 天：PyTorch 张量、CNN 图像分类、目标检测原理、YOLO11/YOLO26 实战、部署与项目整合。

YOLO 更新说明：旧课程中常见 YOLOv3、YOLOv5、Darknet 或早期 Ultralytics YOLOv8 写法已经偏旧。本课件统一更新为 Ultralytics API：

```
from ultralytics import YOLO

model = YOLO("yolo11n.pt")
results = model("image.jpg")
```

实操主线使用 YOLO11，因为资料、生态、任务支持和教学稳定性都比较好；补充介绍 YOLO26，因为截至 2026-06，Ultralytics 官方文档将 YOLO26 标为最新模型，适合讲部署趋势、端侧推理和新模型迁移。

参考资料与 GitHub 课件

类型	地址	用法
Ultralytics 官方仓库	https://github.com/ultralytics/ultralytics	YOLO 新版 API、训练、预测、导出主仓库
YOLO11 官方文档	https://docs.ultralytics.com/models/yolo11/	YOLO11 特性、任务、模型命名、训练预测示例
YOLO26 官方文档	https://docs.ultralytics.com/models/yolo26	最新 YOLO26 模型、端侧部署、NMS-free 趋势
Ultralytics Notebook	https://github.com/ultralytics/ultralytics/blob/main/examples/tutorial.ipynb	官方 Jupyter 教学 Notebook，可改造成课堂实验
DataWhale YOLO Master	https://github.com/datawhalechina/yolo-master	中文 YOLO 系列教学资料，覆盖 YOLOv1-v12、YOLO11 和实操教程
Application of AI YOLO Notebook	https://ezponda.github.io/ai-application-course-materials/notebooks/07_object_detection_with_ultralytics.html	YOLO11 目标检测 Notebook，适合补充课堂演示

8天课程总览

OpenCV + PyTorch CNN + YOLO 8天路线

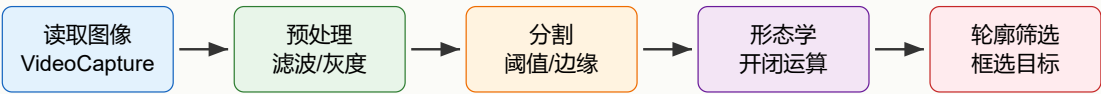


学习路径：传统视觉处理 → CNN 表征学习 → YOLO 目标检测 → OpenCV + YOLO 综合工程

天数	主题	核心能力	课堂产出
Day 1	OpenCV 图像基础	读写图像、像素、颜色空间、绘图	图像读取、颜色转换、绘制标注
Day 2	OpenCV 视频与几何变换	摄像头、视频、缩放、旋转、透视变换	视频读取保存、图像校正脚本
Day 3	OpenCV 分割与轮廓	滤波、阈值、形态学、轮廓检测	建筑物/目标区域提取流程
Day 4	PyTorch 与深度学习基础	张量、自动微分、训练流程	线性回归/简单分类训练脚本
Day 5	CNN 图像分类	卷积、池化、CIFAR10、训练评估	CIFAR10 CNN 分类器
Day 6	目标检测与 YOLO 原理	检测任务、Anchor/Anchor-free、mAP、NMS	YOLO 核心概念图解与指标理解
Day 7	YOLO11 训练与推理	数据集、标注格式、训练、预测、验证	自定义数据集 YOLO11 训练流程
Day 8	建筑物识别综合项目	导出 ONNX、建筑物图片/视频检测、项目封装	OpenCV + YOLO 建筑物识别系统

Day 1 OpenCV 图像基础

OpenCV 传统视觉处理流水线



1.1 学习目标

- 理解图像、像素、分辨率、通道、颜色空间。
- 掌握 `cv2.imread()`、`cv2.imwrite()`、`cv2.imshow()` 的基本使用。
- 掌握 BGR、RGB、GRAY、HSV 的转换。
- 能在图像上绘制矩形、圆、线条和文字，为后续检测结果可视化做准备。

1.2 OpenCV 安装

```
pip install opencv-python opencv-contrib-python matplotlib numpy
```

服务器或无界面环境可用：

```
pip install opencv-python-headless
```

1.3 图像读取、显示与保存

```
import cv2

img = cv2.imread("image.jpg")
print(type(img), img.shape) # H, W, C

cv2.imshow("image", img)
cv2.waitKey(0)
cv2.destroyAllWindows()

cv2.imwrite("output.jpg", img)
```

注意：OpenCV 默认按 BGR 顺序读取图像，Matplotlib 默认按 RGB 显示。如果直接用 Matplotlib 显示 OpenCV 图像，颜色会偏。

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("image.jpg")
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
plt.imshow(rgb)
plt.axis("off")
plt.show()
```

1.4 像素与颜色空间

```
img = cv2.imread("image.jpg")

# 访问第 100 行、第 50 列像素，结果是 BGR
b, g, r = img[100, 50]
print(b, g, r)

# 转灰度
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# 转 HSV，便于颜色阈值分割
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
```

常见颜色空间：

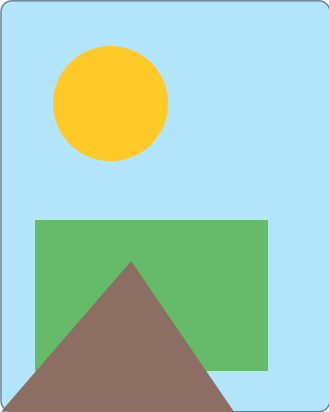
颜色空间	特点	使用场景
BGR	OpenCV 默认格式	图像读取、保存、OpenCV 显示
RGB	通用显示格式	Matplotlib、PIL、网页显示
GRAY	单通道灰度图	边缘检测、阈值分割、轮廓检测
HSV	色相、饱和度、亮度	颜色筛选、目标区域提取
YCrCb	亮度和色度分离	肤色检测、光照鲁棒处理

1.5 绘图与标注

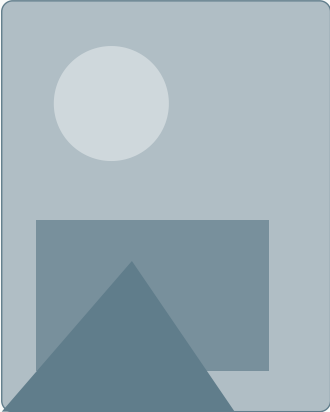
运行效果示意：

运行效果：图像读取、颜色转换与绘图标注

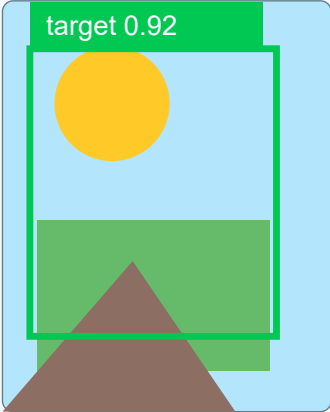
原图 BGR/RGB 显示



灰度图



绘制标注结果



对应代码：cv2.imread、cv2.cvtColor、cv2.rectangle、cv2.putText、cv2.imwrite。

后续 YOLO 检测结果需要画框、画类别、画置信度，因此 OpenCV 绘图必须提前掌握。

```
import cv2

img = cv2.imread("image.jpg")

cv2.rectangle(img, (50, 60), (300, 260), (0, 255, 0), 2)
cv2.circle(img, (175, 160), 40, (0, 0, 255), 2)
cv2.line(img, (50, 60), (300, 260), (255, 0, 0), 2)
cv2.putText(img, "target 0.92", (50, 50), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2)

cv2.imshow("draw", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

1.6 Day 1 实训

任务：读取一张图片，完成以下处理：

1. 显示原图、灰度图、HSV 图。
2. 在原图上画出一个矩形框和类别文字。
3. 保存标注后的图片。

Day 2 OpenCV 视频处理与几何变换

2.1 学习目标

- 掌握摄像头、视频文件读取与保存。
- 掌握图像缩放、平移、旋转、仿射变换、透视变换。
- 理解视频本质：连续图像帧。
- 为后续 YOLO 视频检测打基础。

2.2 摄像头读取

```
import cv2

cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    cv2.imshow("camera", frame)
    if cv2.waitKey(1) & 0xFF == ord("q"):
        break

cap.release()
cv2.destroyAllWindows()
```

2.3 视频文件读取与保存

```
import cv2

cap = cv2.VideoCapture("input.mp4")
fps = cap.get(cv2.CAP_PROP_FPS)
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

fourcc = cv2.VideoWriter_fourcc(*"mp4v")
out = cv2.VideoWriter("output.mp4", fourcc, fps, (width, height))

while True:
    ret, frame = cap.read()
    if not ret:
        break
    out.write(frame)

cap.release()
out.release()
```

2.4 几何变换

```
import cv2
import numpy as np

img = cv2.imread("image.jpg")
h, w = img.shape[:2]

# 缩放
resize = cv2.resize(img, (w // 2, h // 2))

# 平移
M = np.float32([[1, 0, 50], [0, 1, 30]])
move = cv2.warpAffine(img, M, (w, h))

# 旋转
M = cv2.getRotationMatrix2D((w / 2, h / 2), 30, 1.0)
rotate = cv2.warpAffine(img, M, (w, h))
```

2.5 透视变换

透视变换常用于文档矫正、道路俯视图、建筑物立面校正。

```
import cv2
import numpy as np

img = cv2.imread("building.jpg")

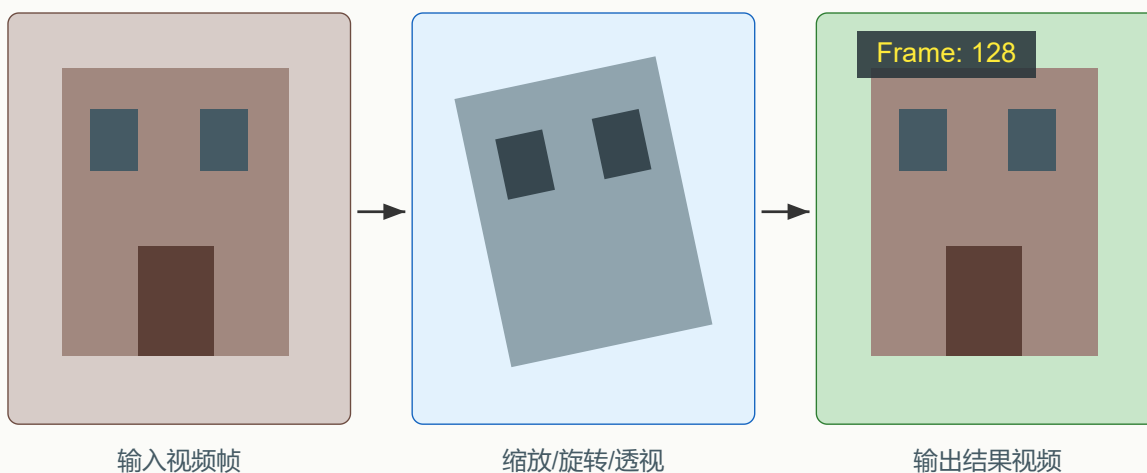
src = np.float32([[120, 80], [520, 100], [560, 420], [90, 430]])
dst = np.float32([[0, 0], [400, 0], [400, 300], [0, 300]])

M = cv2.getPerspectiveTransform(src, dst)
result = cv2.warpPerspective(img, M, (400, 300))
```

2.6 Day 2 实训

运行效果示意：

运行效果：视频逐帧处理与几何变换



对应代码：VideoCapture 读取帧，resize/warpAffine/warpPerspective 处理，VideoWriter 保存。

任务：读取一个视频文件，逐帧处理并保存新视频：

1. 读取每一帧。
2. 缩放到固定大小。
3. 在左上角绘制帧号和 FPS。
4. 保存为新视频。

Day 3 OpenCV 滤波、阈值、形态学与轮廓

3.1 学习目标

- 掌握均值滤波、高斯滤波、中值滤波、双边滤波。
- 掌握全局阈值、自适应阈值、Otsu 阈值。

- 掌握腐蚀、膨胀、开运算、闭运算、形态梯度。
- 掌握轮廓检测、外接矩形、面积筛选。
- 完成一个传统视觉项目流程：建筑物/目标区域提取。

3.2 图像滤波

```
import cv2

img = cv2.imread("image.jpg")

blur = cv2.blur(img, (5, 5))
gaussian = cv2.GaussianBlur(img, (5, 5), 0)
median = cv2.medianBlur(img, 5)
bilateral = cv2.bilateralFilter(img, 9, 75, 75)
```

滤波选择建议：

方法	特点	场景
均值滤波	简单平滑，容易模糊边缘	快速降噪
高斯滤波	按距离加权，更自然	边缘检测前预处理
中值滤波	对椒盐噪声有效	扫描件、噪点图
双边滤波	保边平滑	人脸、美颜、边缘敏感任务

3.3 阈值分割

```
import cv2

img = cv2.imread("image.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

_, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)
adaptive = cv2.adaptiveThreshold(
    gray, 255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    11, 2
)

_, otsu = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
```


3.4 形态学操作

```
import cv2
import numpy as np

kernel = np.ones((5, 5), np.uint8)

erode = cv2.erode(binary, kernel, iterations=1)
dilate = cv2.dilate(binary, kernel, iterations=1)
open_img = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel)
close_img = cv2.morphologyEx(binary, cv2.MORPH_CLOSE, kernel)
gradient = cv2.morphologyEx(binary, cv2.MORPH_GRADIENT, kernel)
```

3.5 轮廓检测

```
import cv2

contours, hierarchy = cv2.findContours(binary, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

for contour in contours:
    area = cv2.contourArea(contour)
    if area < 1000:
        continue

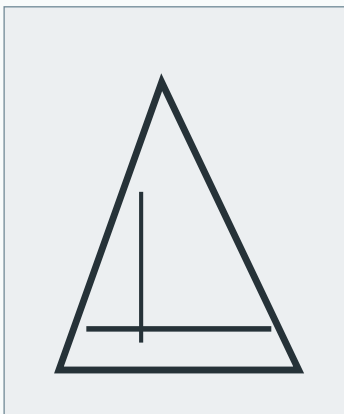
    x, y, w, h = cv2.boundingRect(contour)
    cv2.rectangle(img, (x, y), (x + w, y + h), (0, 255, 0), 2)
```

3.6 项目案例：建筑物识别图像处理流程

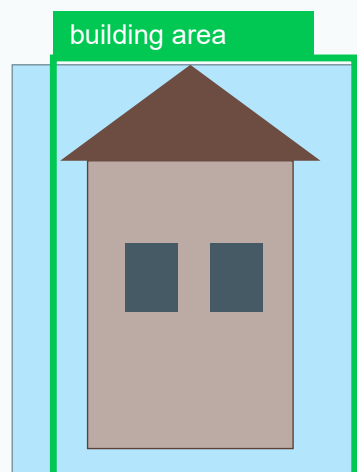
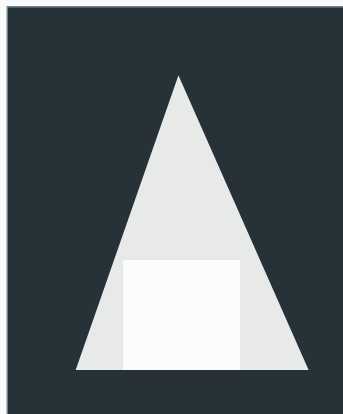
运行效果示意：

运行效果：传统视觉建筑物区域提取

灰度 + 边缘



形态学闭运算



对应代码：GaussianBlur、Canny、morphologyEx、findContours、boundingRect。

传统视觉流程适合做规则明显、光照变化不大的任务。建筑物识别可按如下流程整理：

1. 读取原始图片。
2. 转灰度或 HSV。
3. 根据颜色、亮度或边缘信息做阈值分割。
4. 用开运算去除小噪声。
5. 用闭运算连接断裂区域。
6. 查找轮廓。
7. 根据面积、宽高比、矩形程度筛选建筑区域。
8. 绘制检测框并保存结果。

示例代码：

```
import cv2
import numpy as np

img = cv2.imread("building.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (5, 5), 0)
edge = cv2.Canny(blur, 80, 160)

kernel = np.ones((5, 5), np.uint8)
closed = cv2.morphologyEx(edge, cv2.MORPH_CLOSE, kernel, iterations=2)

contours, _ = cv2.findContours(closed, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

for contour in contours:
    area = cv2.contourArea(contour)
    if area < 2000:
        continue
    x, y, w, h = cv2.boundingRect(contour)
    ratio = w / h
    if 0.3 < ratio < 4:
        cv2.rectangle(img, (x, y), (x + w, y + h), (0, 255, 0), 2)

cv2.imwrite("building_result.jpg", img)
```

3.7 Day 3 实训

任务：完成一个“传统图像处理目标提取”脚本：

- 输入：一张建筑物或目标物图片。
 - 输出：带矩形框的结果图。
 - 要求：包含滤波、阈值/边缘、形态学、轮廓筛选四个步骤。
-

Day 4 PyTorch 与深度学习基础

4.1 学习目标

- 掌握 PyTorch 张量创建、索引、形状变换、设备迁移。
- 理解自动微分、损失函数、优化器。
- 理解深度学习训练流程：数据、模型、损失、优化、评估。

4.2 张量基础

```
import torch

x = torch.tensor([[1, 2], [3, 4]], dtype=torch.float32)
y = torch.zeros((2, 3))
z = torch.randn((4, 5))

print(x.shape)
print(x.reshape(4))
print(x.transpose(0, 1))
```

GPU 使用:

```
device = "cuda" if torch.cuda.is_available() else "cpu"
x = x.to(device)
```

4.3 自动微分

```
import torch

x = torch.tensor(2.0, requires_grad=True)
y = x ** 2 + 3 * x + 1
y.backward()

print(x.grad)
```

4.4 训练流程模板

```
import torch
from torch import nn
from torch.utils.data import DataLoader, TensorDataset

x = torch.randn(1000, 10)
y = (x.sum(dim=1) > 0).long()

dataset = TensorDataset(x, y)
loader = DataLoader(dataset, batch_size=32, shuffle=True)

model = nn.Sequential(
```

```

nn.Linear(10, 32),
nn.ReLU(),
nn.Linear(32, 2)
)

loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

for epoch in range(5):
    for batch_x, batch_y in loader:
        pred = model(batch_x)
        loss = loss_fn(pred, batch_y)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    print(epoch, loss.item())

```

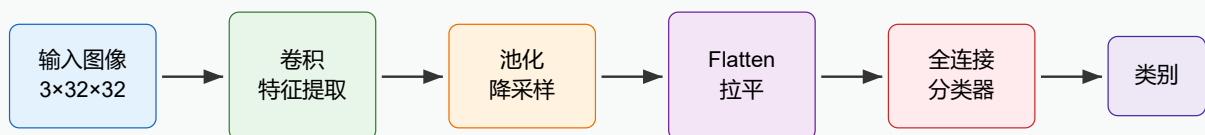
4.5 Day 4 实训

任务：用 PyTorch 完成一个二分类小模型：

1. 构建随机数据集。
2. 定义全连接网络。
3. 完成训练循环。
4. 输出 loss 变化。

Day 5 CNN 图像分类

CNN 图像分类流程



5.1 学习目标

- 理解卷积层、池化层、通道、特征图。
- 掌握 `nn.Conv2d`、`nn.MaxPool2d`、`nn.Flatten`。
- 使用 CIFAR10 完成图像分类训练。

5.2 CNN 核心概念

卷积神经网络适合图像任务，因为它利用了图像的空间结构。卷积核在图像上滑动，提取局部模式；多层卷积逐步从边缘、纹理过渡到物体部件和整体语义。

特征图尺寸计算：

$$\text{输出尺寸} = (\text{输入尺寸} + 2 * \text{padding} - \text{kernel_size}) / \text{stride} + 1$$

5.3 PyTorch CNN 模型

```
import torch
from torch import nn

class SimpleCNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2),
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64 * 8 * 8, 128),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(128, num_classes)
        )

    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x)
        return x
```

5.4 CIFAR10 数据加载

```
import torchvision
from torchvision import transforms
from torch.utils.data import DataLoader

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])
```

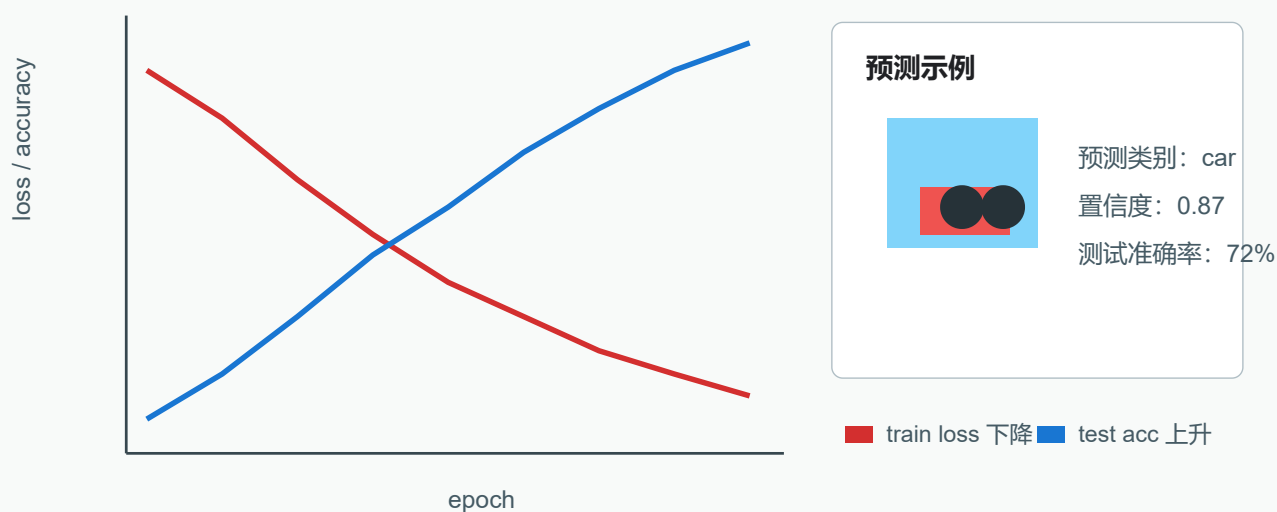
```
train_set = torchvision.datasets.CIFAR10(root="data", train=True, download=True,
transform=transform)
test_set = torchvision.datasets.CIFAR10(root="data", train=False, download=True,
transform=transform)

train_loader = DataLoader(train_set, batch_size=64, shuffle=True)
test_loader = DataLoader(test_set, batch_size=64, shuffle=False)
```

5.5 CNN 训练与评估

运行效果示意：

运行效果：CNN 图像分类训练曲线与预测



对应代码：SimpleCNN、CrossEntropyLoss、Adam、model.train、model.eval、argmax。

```
import torch
from torch import nn

device = "cuda" if torch.cuda.is_available() else "cpu"
model = SimpleCNN().to(device)
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

for epoch in range(10):
    model.train()
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)
        pred = model(images)
        loss = loss_fn(pred, labels)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

```
model.eval()
correct = 0
total = 0
with torch.no_grad():
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)
        pred = model(images)
        correct += (pred.argmax(dim=1) == labels).sum().item()
        total += labels.size(0)

print(f"epoch={epoch}, acc={correct / total:.4f}")
```

5.6 Day 5 实训

任务：训练一个 CIFAR10 分类器，并完成：

- 1. 打印模型结构。
- 2. 输出训练 loss 和测试准确率。
- 3. 保存模型参数。
- 4. 读取一张图片并预测类别。

Day 6 目标检测与 YOLO 原理

6.1 学习目标

- 理解分类、检测、分割的区别。
- 理解目标检测输出：类别、置信度、边界框。
- 理解 IoU、mAP、NMS、Anchor、Anchor-free。
- 理解 YOLO 系列发展脉络。

6.2 图像任务对比

任务	输出	示例
分类	图片属于哪个类别	这张图是猫
检测	目标类别 + 矩形框	图中有 2 个人、1 辆车
实例分割	每个目标的像素级区域	每个人的轮廓区域
语义分割	每个像素属于哪个类别	道路、天空、建筑物
姿态估计	关键点坐标	人体关节点
OBB 旋转框检测	带角度的目标框	航拍船只、遥感建筑物

6.3 检测框与 IoU

IoU 衡量两个框的重叠程度：

$$\text{IoU} = \text{交集面积} / \text{并集面积}$$

```
def box_iou(box1, box2):
    x1 = max(box1[0], box2[0])
    y1 = max(box1[1], box2[1])
    x2 = min(box1[2], box2[2])
    y2 = min(box1[3], box2[3])

    inter = max(0, x2 - x1) * max(0, y2 - y1)
    area1 = (box1[2] - box1[0]) * (box1[3] - box1[1])
    area2 = (box2[2] - box2[0]) * (box2[3] - box2[1])
    union = area1 + area2 - inter
    return inter / union if union > 0 else 0
```

6.4 YOLO 系列更新脉络

版本	教学重点
YOLOv1-v3	YOLO 基本思想、网格预测、Anchor、多尺度特征
YOLOv4-v5	工程化训练技巧、数据增强、CSP、PAN、Mosaic
YOLOv6-v7	工业部署、重参数化、高效检测结构
YOLOv8	Ultralytics 新 API、Anchor-free、Detect/Segment/Pose/Classify 多任务
YOLOv9-v10	训练策略、端到端检测、减少后处理依赖
YOLO11	适合当前教学实操，支持检测、分割、分类、姿态、OBB
YOLO12	Attention-centric 方向，可作为扩展阅读
YOLO26	2026 最新 Ultralytics 模型，强调端侧部署、NMS-free、轻量高效

6.5 YOLO11 与 YOLO26 更新点

YOLO11 适合作为课堂实操主线：

- API 简洁，YOLO("yolo11n.pt") 可直接加载模型。
- 支持检测、实例分割、分类、姿态估计、旋转框检测。
- 模型规模覆盖 n/s/m/l/x，便于按设备性能选择。
- 相比 YOLOv8，官方文档强调更好的特征提取、效率、准确率和多任务适配。

YOLO26 适合作为最新版本趋势：

- 官方资料将其定位为 2026 年最新实时视觉模型。
- 重点面向边缘设备和低功耗部署。

- 强调端到端、NMS-free 推理，减少后处理环节。
- 支持检测、实例分割、语义分割、分类、姿态、旋转框等任务。

教学建议：先用 YOLO11 完成所有训练和项目流程，再用 YOLO26 替换权重文件演示迁移：

```
from ultralytics import YOLO

# 稳定教学主线
model = YOLO("yolo11n.pt")

# 最新版本体验
model = YOLO("yolo26n.pt")
```

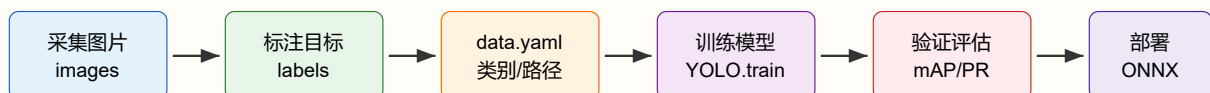
6.6 Day 6 实训

任务：完成目标检测概念练习：

1. 手写 IoU 函数。
2. 理解 mAP、Precision、Recall。
3. 比较分类、检测、分割输出差异。
4. 阅读 YOLO11/YOLO26 模型命名规则。

Day 7 YOLO11 训练、预测与验证

YOLO11/YOLO26 训练与推理流程



课堂主线：YOLO11 完成训练与验证；拓展：YOLO26 替换权重体验最新模型。

7.1 学习目标

- 掌握 Ultralytics 安装与环境检查。
- 掌握 YOLO11 图片预测、视频预测、摄像头预测。
- 掌握自定义数据集 YAML 格式。
- 掌握训练、验证、导出流程。

7.2 安装与环境检查

```
pip install ultralytics
```

```
import ultralytics
ultralytics.checks()
```

7.3 预测图片

```
from ultralytics import YOLO

model = YOLO("yolo11n.pt")
results = model("image.jpg", conf=0.25)

for result in results:
    result.show()
    result.save(filename="predict.jpg")
```

读取检测框：

```
for result in results:
    boxes = result.boxes
    for box in boxes:
        cls_id = int(box.cls[0])
        conf = float(box.conf[0])
        x1, y1, x2, y2 = box.xyxy[0].tolist()
        name = result.names[cls_id]
        print(name, conf, x1, y1, x2, y2)
```

7.4 预测视频与摄像头

```
from ultralytics import YOLO

model = YOLO("yolo11n.pt")

# 视频文件
model.predict(source="input.mp4", save=True, conf=0.25)

# 摄像头
model.predict(source=0, show=True, conf=0.25)
```

7.5 自定义数据集格式

推荐目录：

```
datasets/mydata/
  images/
    train/
    val/
  labels/
    train/
    val/
  data.yaml
```

YOLO 检测标签格式:

```
class_id x_center y_center width height
```

坐标要求归一化到 0-1。

`data.yaml` 示例:

```
path: datasets/mydata
train: images/train
val: images/val

names:
  0: building
  1: car
  2: person
```

7.6 训练 YOLO11

```
from ultralytics import YOLO

model = YOLO("yolo11n.pt")
results = model.train(
    data="datasets/mydata/data.yaml",
    epochs=100,
    imgsz=640,
    batch=16,
    device=0
)
```

命令行写法:

```
yolo detect train model=yolo11n.pt data=datasets/mydata/data.yaml epochs=100 imgsz=640
batch=16
```

7.7 验证与推理

```
metrics = model.val(data="datasets/mydata/data.yaml")
print(metrics.box.map)      # mAP50-95
print(metrics.box.map50)    # mAP50

model.predict(source="test_images", save=True, conf=0.25)
```

7.8 训练参数建议

场景	建议
CPU 或入门电脑	使用 <code>yolo11n.pt</code> , 小数据集, 少量 epoch

场景	建议
单张 8GB 显卡	<code>yolo11n.pt</code> 或 <code>yolo11s.pt</code> , <code>imgsz=640</code>
检测小目标	增大 <code>imgsz</code> , 保证标注质量, 增加小目标样本
数据很少	使用预训练模型, 增加数据增强, 降低学习率
过拟合	增加数据、增强、早停、减小模型规模

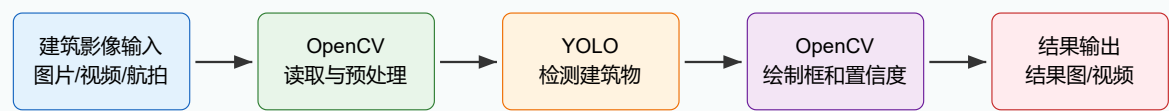
7.9 Day 7 实训

任务：制作一个自定义检测数据集并训练 YOLO11：

1. 准备至少 50 张图片。
2. 用 Labellmg、CVAT、Roboflow 或 X-AnyLabeling 标注。
3. 生成 YOLO 格式标签和 `data.yaml`。
4. 使用 `yolo11n.pt` 训练。
5. 对测试图片进行预测并保存结果。

Day 8 建筑物识别综合项目：OpenCV + YOLO

建筑物识别综合项目架构



工程分工：OpenCV 管输入输出和可视化，YOLO 管建筑物目标检测，配置文件管模型、阈值和保存路径。
适合数据：街景建筑、无人机航拍建筑、工地建筑、遥感建筑切片。

8.1 学习目标

- 理解 YOLO26 和 YOLO11 在教学中的关系。
- 掌握 YOLO 模型导出 ONNX。
- 掌握 OpenCV + YOLO 视频检测项目封装。
- 完成 8 天课程综合项目。

8.2 YOLO26 迁移体验

如果环境中的 `ultralytics` 版本支持 YOLO26，可直接替换模型权重：

```
from ultralytics import YOLO

model = YOLO("yolo26n.pt")
results = model("image.jpg")
```

如果下载失败或当前环境暂不支持，课堂实操仍使用 YOLO11，YOLO26 作为前沿模型介绍即可。

8.3 模型导出

```
from ultralytics import YOLO

model = YOLO("runs/detect/train/weights/best.pt")
model.export(format="onnx", imgsz=640)
```

常见导出格式：

格式	用途
ONNX	跨框架推理，OpenCV DNN、ONNX Runtime
TensorRT	NVIDIA GPU 高性能部署
OpenVINO	Intel CPU/GPU/NPU 部署
CoreML	iOS/macOS
TFLite	Android/边缘设备

8.4 OpenCV + YOLO 视频检测项目

```
import cv2
from ultralytics import YOLO

model = YOLO("yolo11n.pt")
cap = cv2.VideoCapture("input.mp4")

fps = cap.get(cv2.CAP_PROP_FPS)
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fourcc = cv2.VideoWriter_fourcc(*"mp4v")
out = cv2.VideoWriter("yolo_output.mp4", fourcc, fps, (width, height))

while True:
    ret, frame = cap.read()
    if not ret:
        break

    results = model(frame, conf=0.25, verbose=False)
    annotated = results[0].plot()
    out.write(annotated)
```

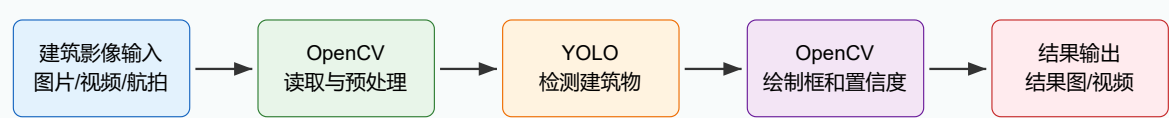
```
cv2.imshow("YOLO", annotated)
if cv2.waitKey(1) & 0xFF == ord("q"):
    break

cap.release()
out.release()
cv2.destroyAllWindows()
```

8.5 综合项目：建筑物识别系统

本课程综合项目是建筑物识别系统。传统 OpenCV 建筑物区域提取和 YOLO 建筑物检测训练都是前置能力，最终工程以“OpenCV 管输入输出，YOLO 管建筑物检测推理”为主线。

建筑物识别综合项目架构



工程分工：OpenCV 管输入输出和可视化，YOLO 管建筑物目标检测，配置文件管模型、阈值和保存路径。
适合数据：街景建筑、无人机航拍建筑、工地建筑、遥感建筑切片。

8.5.1 项目目标

建筑物识别项目适合作为课堂结课项目，因为它能把 8 天内容全部串起来：

课程内容	在建筑物识别项目中的作用
OpenCV 图像基础	读取图片、颜色转换、绘制检测框和文字
OpenCV 视频处理	读取视频/摄像头、逐帧处理、保存视频
OpenCV 传统处理	可作为 ROI 预处理、画面增强、失败案例分析
PyTorch/CNN	理解深度模型如何提取图像特征
YOLO11/YOLO26	完成建筑物目标检测推理和自定义训练
工程封装	参数配置、命令行、输出目录、代码复用

8.5.2 项目目录

本课件目录中已经放好建筑物识别示例代码：

```
opencv_cnn_yolo_8天课程/
  assets/
    course_roadmap.svg
    opencv_pipeline.svg
```

```
cnn_flow.svg
yolo_training_flow.svg
version_c_architecture.svg
code/
  opencv_yolo_project/
    opencv_yolo_detect.py
    config.yaml
    requirements.txt
    README.md
OpenCV_CNN_YOLO_8天合成课件.md
```

代码目录: `code/opencv_yolo_project`

8.5.3 安装依赖

```
cd code/opencv_yolo_project
pip install -r requirements.txt
```

`requirements.txt`:

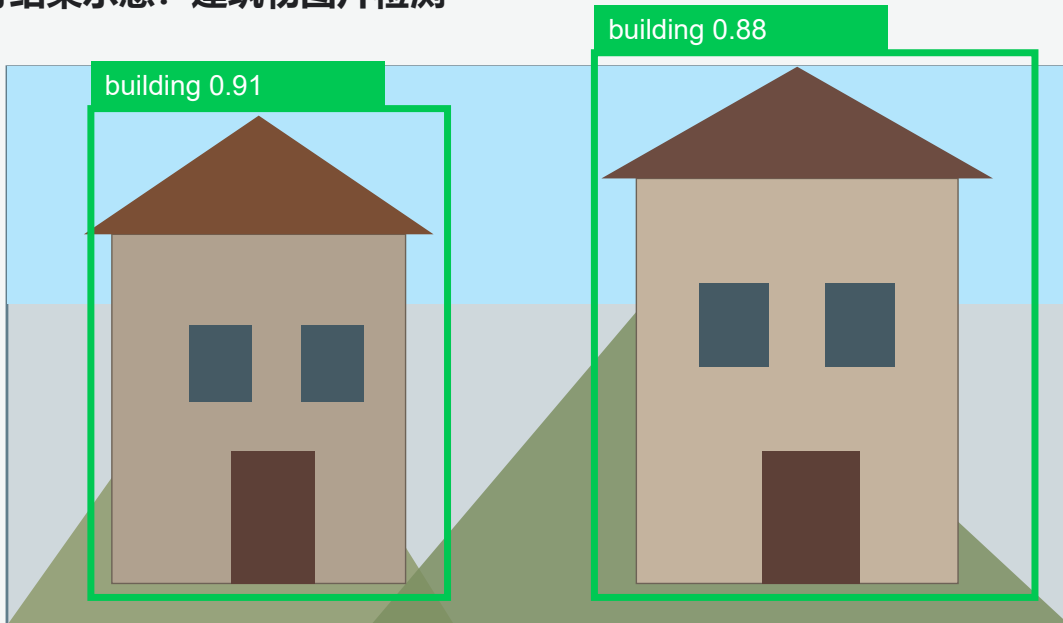
```
ultralytics>=8.3.0
opencv-python>=4.8.0
PyYAML>=6.0
numpy>=1.23
```

8.5.4 图片检测

```
python opencv_yolo_detect.py --source building.jpg --output outputs/building_result.jpg --
model yolo11n.pt
```

运行结果示意:

运行结果示意：建筑物图片检测



示意：程序读取建筑图片后，输出建筑物类别、置信度和矩形框。

流程说明：

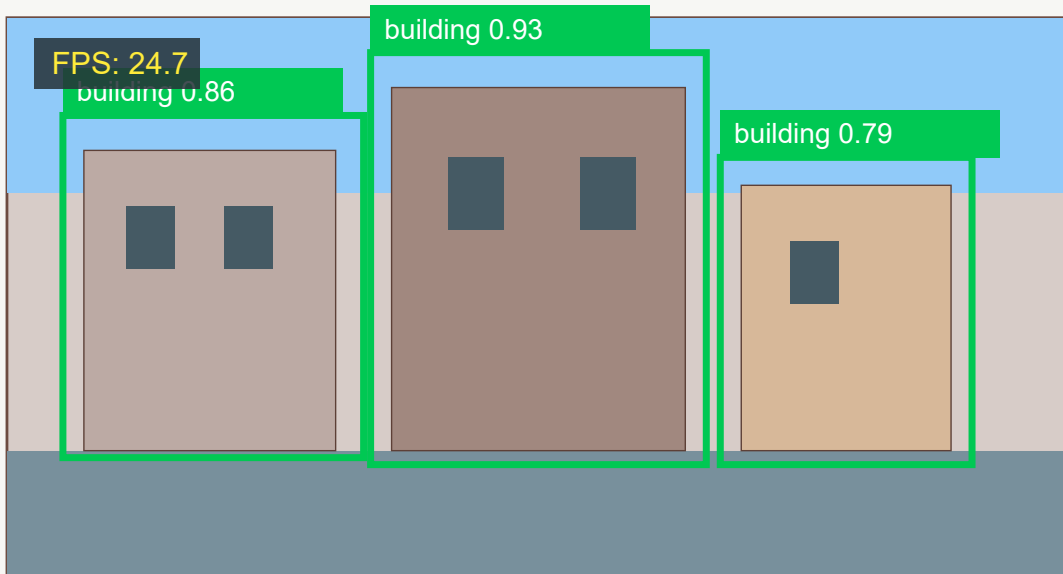
1. OpenCV 使用 `cv2.imread()` 读取图片。
2. YOLO 使用 `model(frame)` 完成检测。
3. OpenCV 使用 `cv2.rectangle()` 和 `cv2.putText()` 绘制结果。
4. OpenCV 使用 `cv2.imwrite()` 保存结果图。

8.5.5 视频检测

```
python opencv_yolo_detect.py --source building_demo.mp4 --output outputs/building_result.mp4  
--model yolo11n.pt
```

运行结果示意：

运行结果示意：建筑物视频检测帧



示意：视频逐帧检测并保存为带检测框的新视频。

视频检测流程：

1. OpenCV 使用 `cv2.VideoCapture()` 打开视频。
2. 循环读取每一帧。
3. YOLO 对当前帧进行检测。
4. OpenCV 绘制检测框、类别、置信度和 FPS。
5. OpenCV 使用 `cv2.VideoWriter()` 写入新视频。

8.5.6 摄像头实时检测

```
python opencv_yolo_detect.py --source 0 --show --model yolo11n.pt
```

按 `q` 可退出实时窗口。

8.5.7 使用配置文件

`config.yaml` 可以统一管理模型、输入源、输出路径、置信度阈值等参数：

```
model: yolo11n.pt
source: input.mp4
output: outputs/result.mp4
conf: 0.25
iou: 0.45
show: false
save: true
classes: []
line_width: 2
```

运行：

```
python opencv_yolo_detect.py --config config.yaml
```

8.5.8 替换 YOLO26

如果当前环境的 Ultralytics 版本已经支持 YOLO26，可以直接替换模型权重：

```
python opencv_yolo_detect.py --source building.jpg --output
outputs/yolo26_building_result.jpg --model yolo26n.pt
```

如果模型下载失败，说明当前环境或镜像源暂不支持对应权重。教学主线继续使用 `yolo11n.pt`，YOLO26 作为最新模型趋势介绍即可。

8.5.9 项目验收标准

项目项	验收要求
图片检测	能读取建筑物图片并保存带检测框的结果图
视频检测	能读取建筑物视频并输出带检测框的新视频
摄像头检测	能实时显示检测结果，按 <code>q</code> 退出
参数配置	能通过命令行或 <code>config.yaml</code> 修改模型、输入源和阈值
代码结构	OpenCV 输入输出、YOLO 推理、绘制函数职责清晰
项目总结	能说明误检、漏检原因，并给出数据和训练改进方案

8.6 Day 8 实训

任务：基于 `code/opencv_yolo_project` 完成建筑物识别端到端视觉项目：

1. 准备建筑物图片或建筑物视频，可以使用街景、工地、航拍或遥感切片。
2. 使用 OpenCV 完成图片/视频读取。
3. 使用 YOLO11 完成建筑物目标检测，OpenCV 完成结果绘制。
4. 保存标注结果图或视频，并提交运行命令和输出文件。
5. 整理训练参数、指标、失败案例和改进方案。